WORKSHOP 研讨/分享/感悟

仓颉社区中的灵感碰撞 / 社区先行者的干货分享

仓颉 Cangjie

毕方AI代码辅助工具介绍

主讲人： FireCoder小方

AI大模型　　实时代码补全

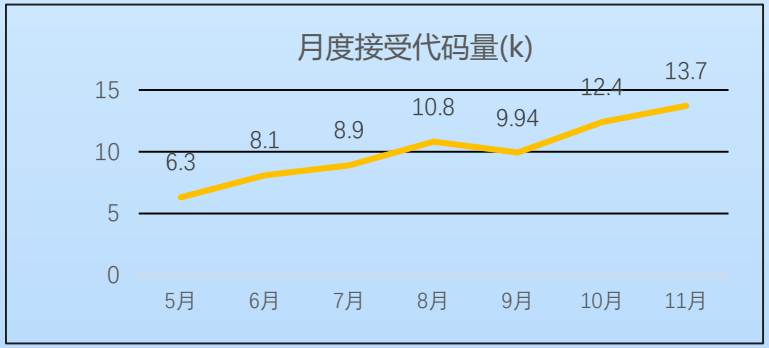


FireCoder：突破数据自演化扩充技术，解决少语料难题，提升代码生成准确率



支撑13家Top应用（力扣、工行、泛微、派电、宝兰德等）、仓颉三方库、仓颉API开发，用户接纳50K+，整体接受率20%+

月度接受代码量趋势



关键技术突破

基于种子数据的自演化扩充：基于程序分析、基于LLM的半自动化数据生成等技术，实现对仓颉现有种子数据的**100倍+**高质量语料扩充

智能熔断：基于IDE编码行为信息，分析用户个性化诉求，滤除低质量请求，并在推理过程中及时清理已过期及无效请求，资源利用率提升**6倍+**

轻量级可控解码：基于确定性前缀、类型/定值分析、作用域分析优化推理候选词列表，从而提升推理生成效果，相对提升评测接纳率**25%+**

仓颉 Cangjie

功能介绍

01 代码补全

02 UT生成

03 代码翻译

毕方AI辅助代码补全，实现自动化编程

功能点：

- 行级和片段级代码实时生成
- 灰度续写
- Token级接受和回退

价值：

- 仓颉更易上手、编码更加轻松，提升研发效率
- 减少重复性开发工作，解放开发者思维



https://gitcode.com/OpenHarmonyToolkitsPlaza/FireCoder/overview

仓颉 Cangjie

毕方AI辅助代码补全——效果演示

```
public static func onNetworkStateChange(options: ObserverOptions, callback: Callback1Argument<NetworkState>): Unit {
    synchronized(onOffMutex) {
        for (item in callbackList_d) {
            if (refEq(callback, item[0]) && item[1] == "networkStateChange") {
                Telephony_ObserverLog.error("Telephony Observer on networkStateChange failed: The same function has registered.
                return
            }
        }
        unsafe {
            let wrapper = {value: CNetworkState => callback.invoke(value.toNetworkState())}
            let lambdaData = Callback1Param<CNetworkState, Unit>(wrapper)
            let errCode = FfiTelephonyObserverOnNetworkStateChange(options, lambdaData.getID())
            if (errCode != SUCCESS_CODE) {
                Telephony_ObserverLog.info("Failed to on networkStateChange. code: ${errCode}, ${getErrorMsg(errCode)}")
                throw BusinessException(errCode, "Telephony Observer on networkStateChange failed: ${getErrorMsg(errCode)}")
            }
            callbackList_d.append((callback, "networkStateChange", lambdaData.getID()))
        }
    }
}
```

上文

```
public static func onSignalInfoChange(options: ObserverOptions, callback: Callback1Argument<SignalInformation>): Unit {
    synchronized(onOffMutex) {
        for (item in callbackList_d) {
            if (refEq(callback, item[0]) && item[1] == "signalInfoChange") {
                Telephony_ObserverLog.error("Telephony Observer on signalInfoChange failed: The same function has registered.
                return
            }
        }
        unsafe {
            let wrapper = {value: CSignalInformation => callback.invoke(value.toSignalInformation())}
            let lambdaData = Callback1Param<CSignalInformation, Unit>(wrapper)
            let errCode = FfiTelephonyObserverOnSignalInfoChange(options, lambdaData.getID())
            if (errCode != SUCCESS_CODE) {
                Telephony_ObserverLog.info("Failed to on signalInfoChange. code: ${errCode}, ${getErrorMsg(errCode)}")
                throw BusinessException(errCode, "Telephony Observer on signalInfoChange failed: ${getErrorMsg(errCode)}")
            }
            callbackList_d.append((callback, "signalInfoChange", lambdaData.getID()))
        }
    }
}
```

毕方AI工具助力仓颉代码自动生成

```
                funcId = item[2]
                break
            }
            i++
        }
        if (funcId == 0) {
            Telephony_ObserverLog.error("Telephony Observer off signalInfoChange: This function cannot be found.")
            return
        }
        unsafe {
            let errCode = FfiTelephonyObserverOffSignalInfoChange(funcId)
            if (errCode != SUCCESS_CODE) {
                Telephony_ObserverLog.info("Failed to off signalInfoChange. code: ${errCode}, ${getErrorMsg(errCode)}")
                throw BusinessException(errCode, "Telephony Observer off networkSsignalInfoChangetateChange failed: ${getErrorMsg(
            }
            callbackList_d.remove(i)
        }
    }
}

public static func offSignalInfoChange(): Unit {
    synchronized(onOffMutex) {
        var i = 0
        for (item in callbackList_d) {
            if (item[1] == "signalInfoChange") {
                callbackList_d.remove(i)
            }
            i++
        }
        unsafe {
            let errCode = FfiTelephonyObserverOffAllSignalInfoChange()
            if (errCode != SUCCESS_CODE) {
                Telephony_ObserverLog.info("Failed to off all signalInfoChange. code: ${errCode}, ${getErrorMsg(errCode)}")
                throw BusinessException(errCode, "Telephony Observer off all signalInfoChange failed: ${getErrorMsg(errCode)}")
            }
        }
    }
}

public static func onCallStateChange(options: ObserverOptions, callback: Callback1Argument<CallStateInfo>): Unit {
```

仓颉 Cangjie

AI赋能的智能UT生成，实现AI辅助DT编码增效

功能点：

- 项目级、文件级、函数级UT生成

价值：

- 快速生成测试用例，减少手动编写的时间和精力，提高开发效率
- 全面覆盖，识别边界，提高测试覆盖率、测试质量

```
package ohos_app_cangjie_sha1

import std.collection.*

public class Sha1 {
    private let EXTRA: Array<Int64> = [-2147483648, 8388608, 32768, 128]
    private let SHIFT: Array<Int64> = [24, 16, 8, 0]
    private let HEX_CHARS: Array<Rune> = "0123456789abcdef".toRuneArray()
    private let INPUT_ERROR: String = "input is invalid type"

    public var blocks: ArrayList<Int64>
    public var h0: Int64
    public var h1: Int64
    public var h2: Int64
    public var h3: Int64
    public var h4: Int64
    public var block: Int64
    public var start: Int64
    public var bytes: Int64
    public var hBytes: Int64
    public var finalized: Bool
    public var hashed: Bool
    public var first: Bool
    public var lastByteIndex: Int64

    public init() {...}

    private func safeAdd(a: Int64, b: Int64): Int64 {
        let lsw = (a & 0xFFFF) + (b & 0xFFFF)
        let msw = (a >> 16) + (b >> 16) + (lsw >> 16)
        return ((msw << 16) | (lsw & 0xFFFF)) & 0xFFFFFFFF
    }

    private func safeSubtract(a: Int64, b: Int64): Int64 {
        return safeAdd(a, -b) & 0xFFFFFFFF
    }

    private func custom_add(value1: Int64, value2: Int64, value3: Int64, value4: Int64, value5: Int64): Int64 {...}

    private func custom_subtract(value1: Int64, value2: Int64, value3: Int64, value4: Int64, value5: Int64): Int64 {...}

    private func unsignedRightShift(value: Int64, shift: Int64): Int64 {...}

    public func update(message: String): Sha1 {...}

    public func finalizeHash() {...}

    private func hash() {...}

    public func hex(): String {...}

    public func arrayBuffer(): Array<Byte> {...}

    public func digest(): Array<Byte> {...}

    public func array(): Array<Byte> {...}

    public func toString(): String {
        return hex()
```

```
package ohos_app_cangjie_sha1

import std.collection.*
import std.unittest.*
import std.unittest.testmacro.*

@Test
public class TestSha1 {
// Skip Generating - UUT is private func: ('C:\\Users\\l00566851\\DevEcoStudioProjects\\sha1_s\\sha1\\src\\main\\cangjie\\src\\sha
// Skip Generating - UUT is private func: ('C:\\Users\\l00566851\\DevEcoStudioProjects\\sha1_s\\sha1\\src\\main\\cangjie\\src\\sha
// Skip Generating - UUT is private func: ('C:\\Users\\l00566851\\DevEcoStudioProjects\\sha1_s\\sha1\\src\\main\\cangjie\\src\\sha
// Skip Generating - UUT is private func: ('C:\\Users\\l00566851\\DevEcoStudioProjects\\sha1_s\\sha1\\src\\main\\cangjie\\src\\sha
// Skip Generating - UUT is private func: ('C:\\Users\\l00566851\\DevEcoStudioProjects\\sha1_s\\sha1\\src\\main\\cangjie\\src\\sha
// Skip Generating - UUT is private func: ('C:\\Users\\l00566851\\DevEcoStudioProjects\\sha1_s\\sha1\\src\\main\\cangjie\\src\\sha
    @TestCase
    func testSha1Update(): Unit {
        var sha1 = Sha1()
        var str = "hello"
        var res = sha1.update(str)
        @Assert(res.toString(), "hello")
    }

    @TestCase
    func testSha1Finalizehash(): Unit {
        var sha1 = Sha1()
        var str = "hello world"
        var res = sha1.update(str).finalizeHash()
        var exp : Array<UInt8> = [198, 179, 102, 104, 198, 179, 102, 104, 198, 179, 102, 104, 198, 179, 102, 104, 198, 179, 102, 1
        @Assert(sha1.arrayBuffer(), exp)
    }

    @TestCase
    func testSha1Hex(): Unit {...}

    @TestCase
    func testSha1Arraybuffer(): Unit {...}

    @TestCase
    func testSha1Digest(): Unit {...}

    @TestCase
    func testSha1Array(): Unit {...}

    @TestCase
    func testSha1ToString(): Unit {
        var sha1 = Sha1()
        var str = "hello world"
        sha1.update(str)
        sha1.finalizeHash()
        var res = sha1.toString()
        var exp = "64ec88ca00b268e5ba1a35678a1b5319016a767c"
        @Assert(res, exp)
    }
}
```

代码翻译，实现高效、准确、自动化代码转换

功能点：

- 项目级、文件级、函数级代码翻译

价值：

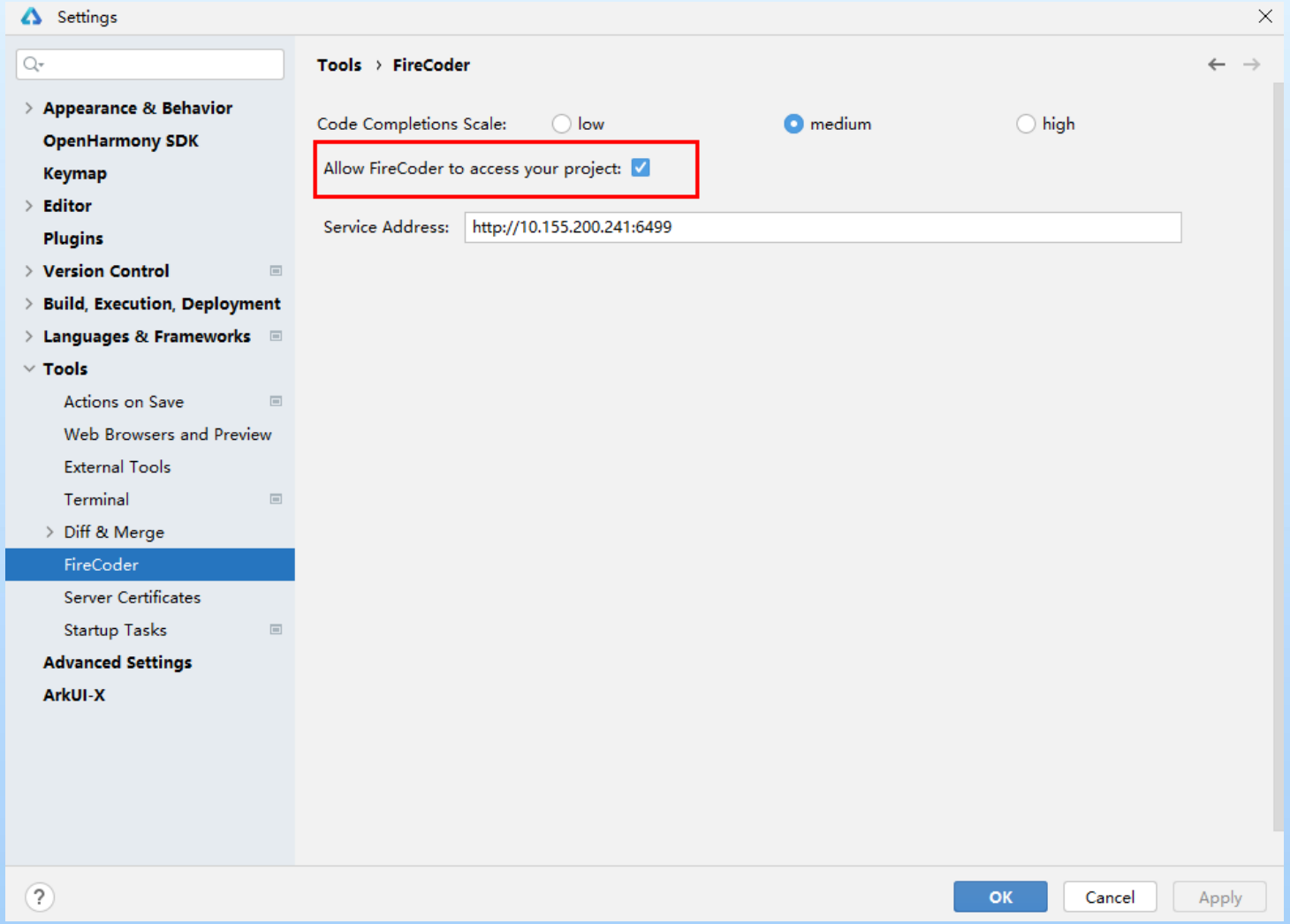
- 减少从头编写cangjie代码工作量，降低开发成本
- 自动化快速迁移，减少重复劳动，提高开发效率

```
public func testcase(done: (Unit) -> Unit): Unit {
    let caseName = "createColorPickerTest002"
    let msgStr = "jsunittest effectKitTest" + (caseName) + " "
    println((msgStr) + "begin")
    let imageSource = image.createImageSource(testPng.buffer)
    imageSource.getImageInfo((err, value) => {
        if (err) {            try {
                expect(true).assertFail()
            } catch (err: Exception) {
                println((msgStr) + "expect assert fail")
            }
            done()
            return
        }
        println((msgStr) + "Napi size=" + value.size.width.toString() + " X " + value.size.height.toString())
        let size: image_Size = image_Size(value.size.width, height: value.size.height)
        println((msgStr) + "promise create pixmap.start")
        imageSource.createPixelMap([image_DownloadOption.desiredSize.set(value.size)]).then(pixelMap => {
            println((msgStr) + "Napi create ColorPicker.start")
            effectKit.createColorPicker(pixelMap, [0.1, 0.1, 0.5, 0.8]).then(colorPicker => {
                println((msgStr) + "Napi createColorPicker")
                let color = colorPicker.getAverageColor()
                println(msgStr + JSON.toString(color))
                expect(color.alpha.isSome()).assertTrue()
                expect(color.red.isSome()).assertTrue()
                expect(color.green.isSome()).assertTrue()
                expect(color.blue.isSome()).assertTrue()
                done()
            }).catch((ex: BusinessError) => {
                println((msgStr) + "Napi colorPicker.error=" + ex.toString())
                expect(true).assertFail()
                done()
            })
        }).catch((ex: BusinessError) => {
            println((msgStr) + "Napi pixelMap.error=" + ex.toString())
            expect(true).assertFail()
            done()
        })
    })
}
```

```
@TestCase
func createColorPickerTest002() {
    let caseName = "createColorPickerTest002"
    let msgStr = "jsunittest effectKitTest" + (caseName) + " "
    logger.info((msgStr) + "begin")
    let imageSource = image.createImageSource(testPng.buffer)
    comment("imageSource.getImageInfo").assertAsyncCallBackWouldBeCalledIn(1.second) { asyncCallBackEvent =>
        imageSource.getImageInfo() { err, _ =>
            asyncCallBackEvent.happens()
            if (err) {            try {
                    comment("expect").assertFalse(trueassertFalse)
                } catch (err: Exception) {
                    logger.info((msgStr) + "expect assert fail")
                }
                waitForAllAssertionsToComplete()
                return
            }
            logger.info((msgStr) + "size=" + value.size.width.toString() + " X " + value.size.height.toString())
            let size: image_Size = image_Size(value.size.width, height: value.size.height)
            logger.info((msgStr) + "promise create pixmap.start")
            try {
                comment("imageSource.createPixelMap").assertAsyncCallBackWouldBeCalledIn(1.second) { asyncCallBackEvent =>
                    imageSource.createPixelMap([image_DownloadOption.desiredSize.set(value.size)]) { error, data =>
                        asyncCallBackEvent.happens()
                        match (data) {
                            case None => ()
                            case Some(pixelMap) => {
                                logger.info((msgStr) + "create ColorPicker.start")
                                try {
                                    comment("effectKit.createColorPicker").assertAsyncCallBackWouldBeCalledIn(1.second) { asyncCallBackEvent =>
                                        effectKit.createColorPicker(pixelMap, [0.1, 0.1, 0.5, 0.8]) { error, data =>
                                            asyncCallBackEvent.happens()
                                            match (data) {
                                                case None => ()
                                                case Some(colorPicker) => {
                                                    logger.info((msgStr) + "createColorPicker")
                                                    let color = colorPicker.getAverageColor()
                                                    logger.info(msgStr + JSON.color.toString())
                                                    comment("expect").assertTrue(color.alpha !== nullassertTrue)
                                                    comment("expect").assertTrue(color.red !== nullassertTrue)
                                                    comment("expect").assertTrue(color.green !== nullassertTrue)
                                                    comment("expect").assertTrue(color.blue !== nullassertTrue)
                                                    waitForAllAssertionsToComplete()
                                                }
                                            }
                                        }
                                    }
                                } catch (ex: Exception) {
                                    logger.info((msgStr) + "colorPicker.error=" + ex.toString())
                                    comment("expect").assertFalse(trueassertFalse)
                                    waitForAllAssertionsToComplete()
                                }
                            }
                        }
                    }
                }
            }
        }
    }
```

重点说明

- 当前该AI工具模型部署在华为云服务器，使用该工具，会将本地项目上传到华为云服务器。



https://gitcode.com/OpenHarmonyToolkitsPlaza/FireCoder/overview

仓颉 Cangjie

WORKSHOP 研讨/分享/感悟

仓颉社区中的灵感碰撞 / 社区先行者的干货分享

仓颉 Cangjie

感谢您的观看!

鸿蒙突击队

FireCoder工具下载链接:
https://gitcode.com/OpenHarmonyToolkitsPlaza/FireCoder/overview

